LAB ASSIGNMENT 11

Problem 1:

Given a pointer to the root of a binary tree, print the top view of the binary tree. The tree as seen from the
top the nodes, is called the top view of the tree.

SOL:

```
//topview of a binary tree
#include<iostream>
using namespace std;

struct Node{
   int data;
   Node *left,*right;
};

Node* create(int x){
   Node* n=new Node;
   n->data=x;
   n->left=n->right=NULL;
   return n;
}

int a[100],vis[100]={0},level[100];

void topview(Node* root,int h,int d){
   if(root==NULL) return;

   if(vis[h+50]==0 || d<level[h+50]){
      a[h+50]=root->data;
      level[h+50]=d;
      vis[h+50]=1;
   }
   topview(root->left,h-1,d+1);
   topview(root->right,h+1,d+1);
}

int main(){
   Node* root=create(1);
   root->left=create(2);
   root->right=create(3);
   root->left->left=create(4);
   root->left->right=create(5);

   topview(root,0,0);

   for(int i=0;i<100;i++){
      if(vis[i])
```

```
        cout<<a[i]<<" ";
    }
    return 0;
}
```

OUTPUT:

```
4 2 1 3
```

Problem 2:

A university maintains student records in a Binary Search Tree (BST), where each node stores a unique
student ID. Due to a software malfunction, the values of exactly two nodes have been swapped. The tree
structure remains unchanged.
Write a C program to recover the original BST by correcting the two swapped values without modifying the
tree structure.

SOL:

```
#include<iostream>
using namespace std;

struct Node{
    int data;
    Node *left,*right;
    Node(int x){
        data=x;
        left=right=NULL;
    }
};

Node *first=NULL,*middle=NULL,*last=NULL,*previous=NULL;

void recover(Node* root){
    if(root==NULL)return;
    recover(root->left);

    if(previous && root->data<previous->data){
        if(first==NULL){
            first=previous;
            middle=root;
        }else last=root;
    }
```

```
    previous=root;
    recover(root->right);
}

void inorder(Node* root){
    if(root==NULL)return;
    inorder(root->left);
    cout<<root->data<<" ";
    inorder(root->right);
}

int main(){
    Node* root=new Node(6);
    root->left=new Node(10);
    root->right=new Node(2);
    root->left->left=new Node(1);
    root->left->right=new Node(3);
    root->right->right=new Node(12);

    recover(root);

    if(last){
        int t=first->data;
        first->data=last->data;
        last->data=t;
    }else if(first && middle){
        int t=first->data;
        first->data=middle->data;
        middle->data=t;
    }

    cout<<"Recovered BST: ";
    inorder(root);
    return 0;
}
```

OUTPUT:

```
Recovered BST: 1 2 3 6 10 12 %
```

Problem 3:

A navigation application stores location codes in a BST. Given a target value x and an integer k, the application must retrieve the k stored values closest to x. Write a C program to return the k values having the smallest absolute differences from x. If two values have the same difference, select the smaller value first..

Sol:

```
//kth closest to root in binary tree
#include<iostream>
using namespace std;

struct Node{
   int data;
   Node *left,*right;
};

Node* create(int x){
   Node* n=new Node;
   n->data=x;
   n->left=n->right=NULL;
   return n;
}

Node* p[100],*s[100];
int pt=-1,st=-1;

void pred(Node* r,int x){
   while(r){
      if(r->data<=x){
         p[++pt]=r;
         r=r->right;
      }else r=r->left;
   }
}

void succ(Node* r,int x){
   while(r){
      if(r->data>x){
         s[++st]=r;
         r=r->left;
      }else r=r->right;
   }
}

int main(){
   Node* r=create(10);
   r->left=create(5);
   r->right=create(15);
   r->left->left=create(2);
   r->left->right=create(7);
   r->right->left=create(12);
   r->right->right=create(20);

   int x=9,k=4;
   pred(r,x);
```

```
    succ(r,x);

    for(int i=0;i<k;i++){
        if(pt==-1){
            cout<<s[st]->data<<" ";
            Node* n=s[st--];
            for(n=n->right;n;n=n->left)s[++st]=n;
        }else if(st==-1){
            cout<<p[pt]->data<<" ";
            Node* n=p[pt--];
            for(n=n->left;n;n=n->right)p[++pt]=n;
        }else if(x-p[pt]->data<=s[st]->data-x){
            cout<<p[pt]->data<<" ";
            Node* n=p[pt--];
            for(n=n->left;n;n=n->right)p[++pt]=n;
        }else{
            cout<<s[st]->data<<" ";
            Node* n=s[st--];
            for(n=n->right;n;n=n->left)s[++st]=n;
        }
    }
    return 0;
}
```

OUTPUT:

```
  10 7 12 5 %
```

Problem4:

A financial analytics system represents a hierarchical portfolio using a binary tree. Each node contains the
net value of a portfolio component. Some subtrees satisfy the BST property, while others do not. Write a C program to find the maximum sum of node values among all subtrees that are valid BSTs. A subtree is a valid BST if every value in its left subtree is strictly smaller than its root and every value in
its right subtree is strictly greater than its root.

Sol:

```
#include<iostream>
#include<climits>
using namespace std;

struct Node{
```

```
    int data;
    Node *left,*right;
    Node(int x){
        data=x;
        left=right=NULL;
    }
};

int ans=0;

int solve(Node* r,int &mn,int &mx){
    if(r==NULL){
        mn=INT_MAX;
        mx=INT_MIN;
        return 0;
    }

    int lmin,lmax,rmin,rmax;
    int l=solve(r->left,lmin,lmax);
    int rr=solve(r->right,rmin,rmax);

    if(r->data>lmax && r->data<rmin){
        mn=min(r->data,lmin);
        mx=max(r->data,rmax);
        int sum=l+rr+r->data;
        ans=max(ans,sum);
        return sum;
    }

    mn=INT_MIN;
    mx=INT_MAX;
    return 0;
}

int main(){
    Node* root=new Node(1);
    root->left=new Node(4);
    root->right=new Node(3);
    root->left->left=new Node(2);
    root->left->right=new Node(4);
    root->right->left=new Node(2);
    root->right->right=new Node(5);
    root->right->right->right=new Node(6);

    int mn,mx;
    solve(root,mn,mx);

    cout<<"Maximum sum BST: "<<ans;
```

```
    return 0;
}
```

OUTPUT:



Problem 5:

IA distributed application needs to transmit a binary tree from one server to another. Since pointers cannot
be transmitted directly, the tree must be converted into a sequence of values and subsequently reconstructed.
Write a C program implementing the following functions:
• serialize(root): Converts a binary tree into a string or sequence.
• deserialize(data): Reconstructs the original binary tree from that representation

SOL:

```
#include<iostream>
#include<sstream>
using namespace std;

struct Node{
    int data;
    Node *left,*right;
    Node(int x){
        data=x;
        left=right=NULL;
    }
};

void save(Node* r,string &s){
    if(r==NULL){
        s+="# ";
        return;
    }
    s+=to_string(r->data)+" ";
    save(r->left,s);
    save(r->right,s);
}

Node* load(stringstream &ss){
    string s;
    ss>>s;
```

```
    if(s=="#")return NULL;
    Node* r=new Node(stoi(s));
    r->left=load(ss);
    r->right=load(ss);
    return r;
}

void inorder(Node* r){
    if(r==NULL)return;
    inorder(r->left);
    cout<<r->data<<" ";
    inorder(r->right);
}

int main(){
    Node* r=new Node(1);
    r->left=new Node(2);
    r->right=new Node(3);
    r->left->left=new Node(4);
    r->left->right=new Node(5);

    string s="";
    save(r,s);
    cout<<"Serialized: "<<s<<endl;

    stringstream ss(s);
    Node* r2=load(ss);

    cout<<"Tree after deserialization: ";
    inorder(r2);
    return 0;
}
```

OUTPUT:

```
Serialized: 1 2 4 # # 5 # # 3 # #
Tree after deserialization: 4 2 5 1 3
```